使用Buildroot制作根文件系统

作者：百问网韦东山

参考文档：https://buildroot.org/downloads/manual/manual.html

根据这几个问题理解Buildroot：

- 如何决定是否编译某些包：
  - make menuconfig
  - 菜单内容来自哪里？
- 如何下载这些包
  - 每个包，应该提供自己的下载地址
- 如何编译这些包
  - 每个包，应该提供自己的编译方法
- 如何制作映像文件
  - 这些被选择的包，被编译后，放入一个统一的目录
  - 制作为rootfs映像
  - 这还不够，我们想制作emmc的映像文件：它含有分区表、根文件系统分区、其他分区

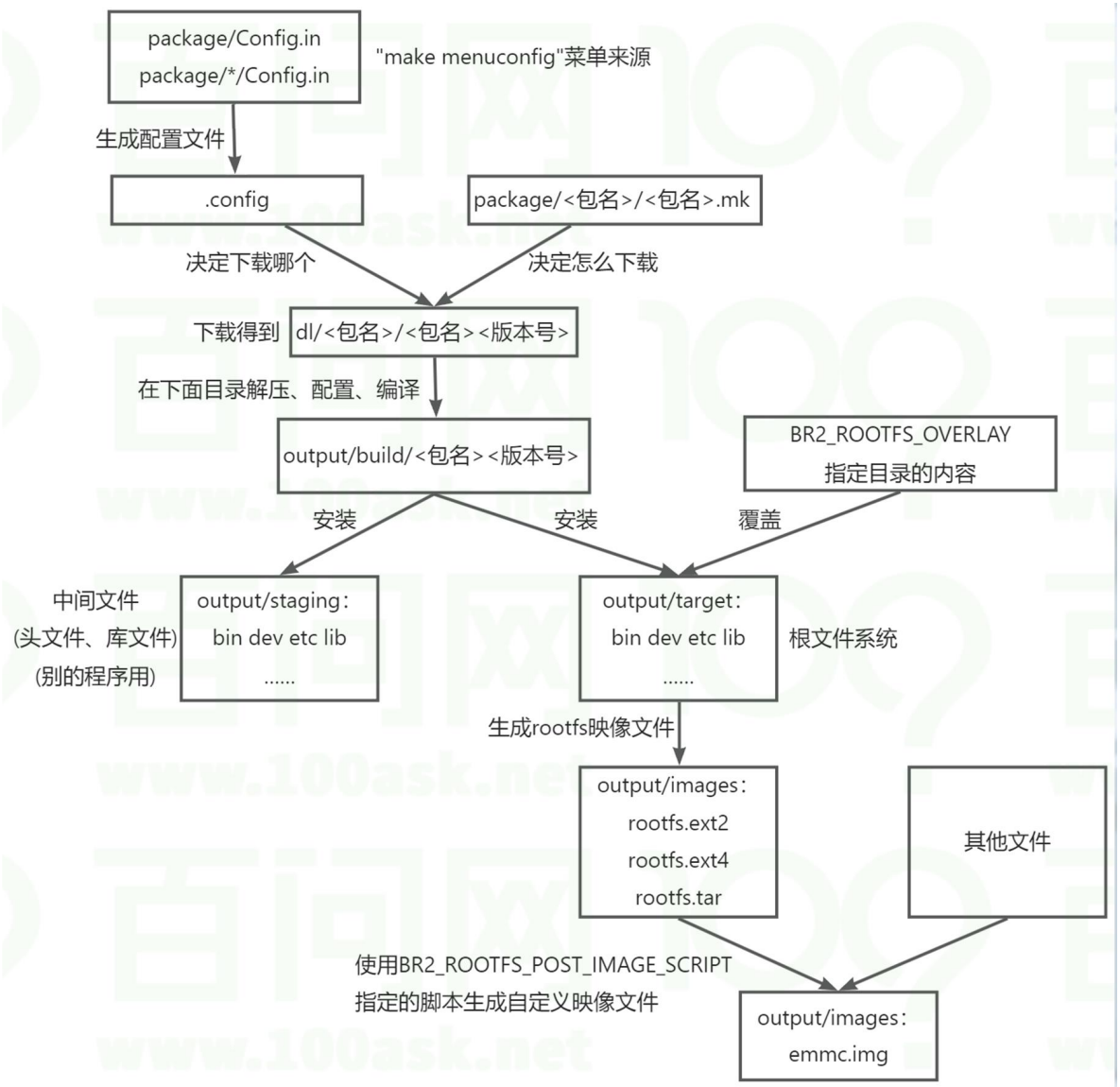
1. Buildroot目录

1.1 目录结构

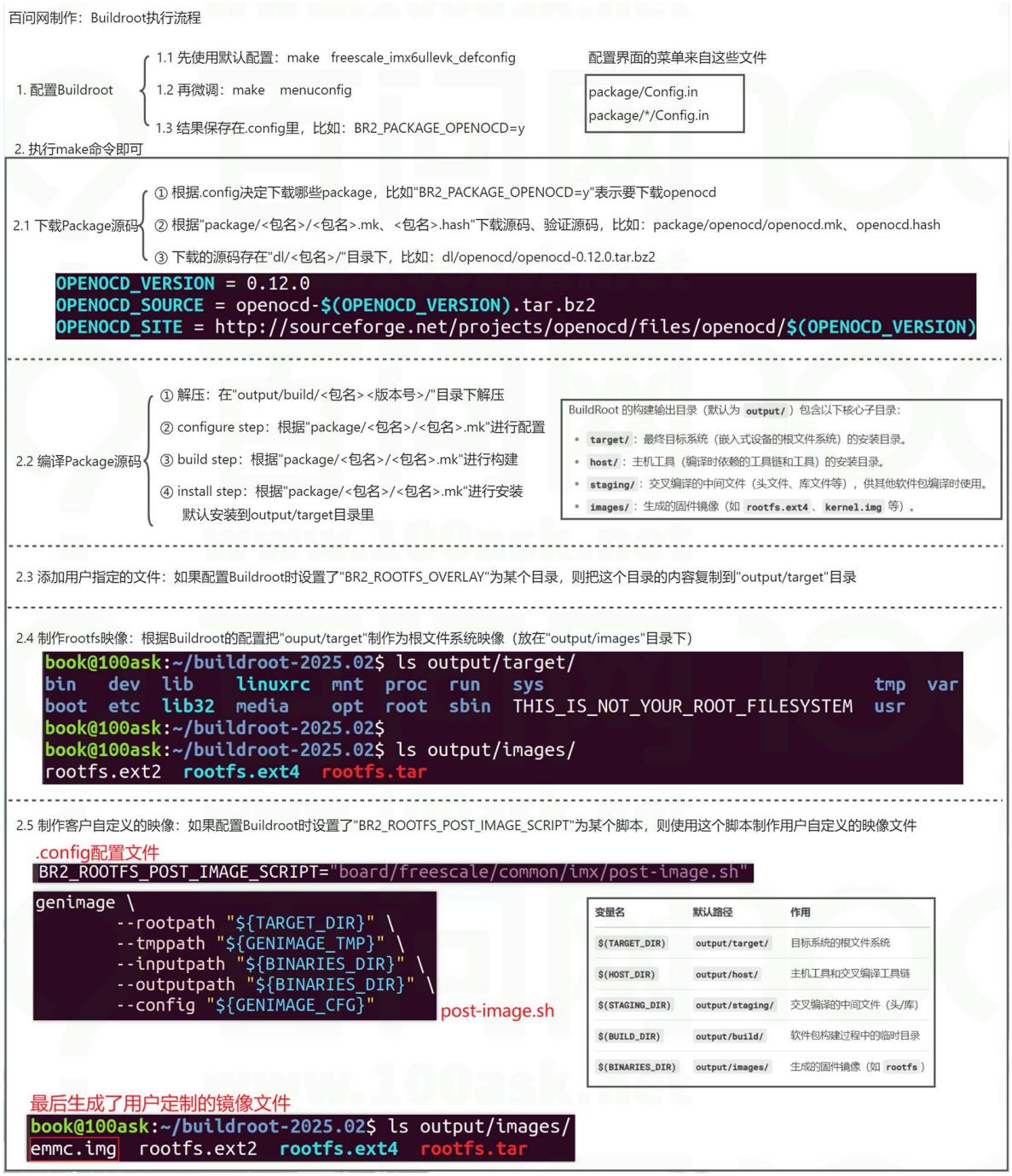
百问网制作：Buildroot目录结构



1.2 执行过程中目录的使用



2. Buildroot执行过程

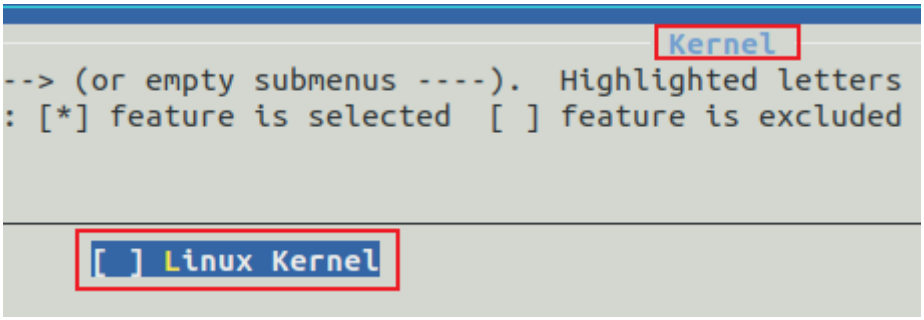


3. 使用默认配置编译根文件系统

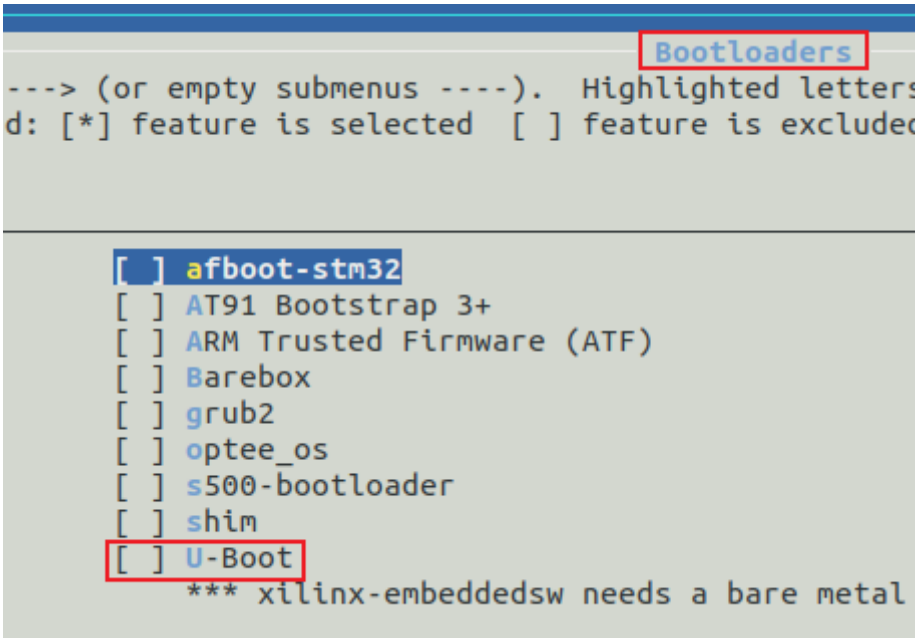
3.1 先使用默认配置与脚本

```
wget https://buildroot.org/downloads/buildroot-2025.02.tar.xz
tar xJf buildroot-2025.02.tar.xz
cd buildroot-2025.02
ls configs/*imx6ull*
make freescale_imx6ullevk_defconfig
make menuconfig # 去掉内核、uboot，方法如下
make -j 4
```

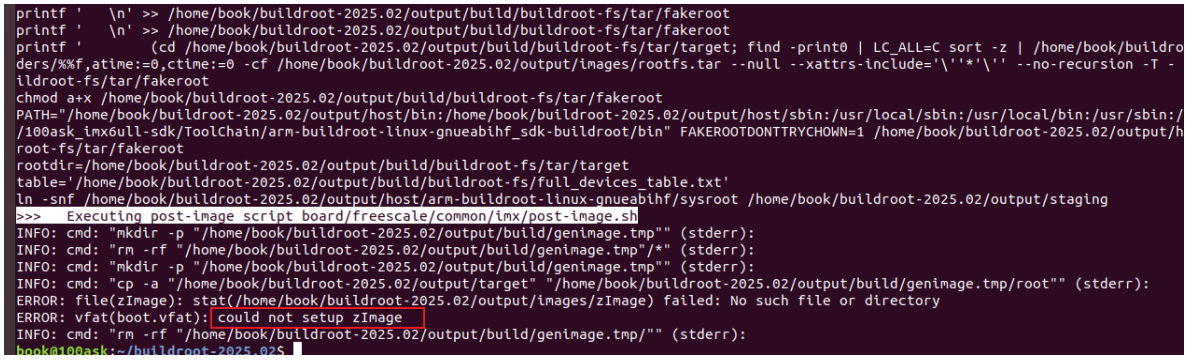
去掉内核：



去掉uboot：



编译最后，会使用`board/freescale/common/imx/post-image.sh`来生成映像文件，有如下错误：



3.2 添加、修改脚本

新建文件`board/freescale/100ask/genimage.cfg`：

```
image  emmc.img {
  hdimage {               // Generates DOS partition images.

  }

  partition rootfs-1 {
    partition-type = 0xC
```

```
    bootable = "yes"
    size = 10M
  }
  partition rootfs-2 {
    partition-type = 0x83
    bootable = "yes"
    image = "rootfs.ext4"
    size = 1000M
  }
}
```

修改`board/freescale/common/imx/post-image.sh`，内容如下：

```
main()
{
    local FILES IMXOFFSET UBOOTBIN GENIMAGE_CFG GENIMAGE_TMP
    FILES="$(dtb_list) $(linux_image)"
    IMXOFFSET="$(imx_offset)"
    UBOOTBIN="$(uboot_image)"
    GENIMAGE_CFG="$(mktemp --suffix genimage.cfg)"
    GENIMAGE_TMP="${BUILD_DIR}/genimage.tmp"

    sed -e "s/%FILES%/${FILES}/" \
        -e "s/%IMXOFFSET%/${IMXOFFSET}/" \
        -e "s/%UBOOTBIN%/${UBOOTBIN}/" \
        "board/freescale/common/imx/$(genimage_type)" > "${GENIMAGE_CFG}"

    rm -rf "${GENIMAGE_TMP}"

    cp board/freescale/100ask/genimage.cfg "${GENIMAGE_CFG}"
    cat "${GENIMAGE_CFG}"
```

3.3 增加内核与设备树

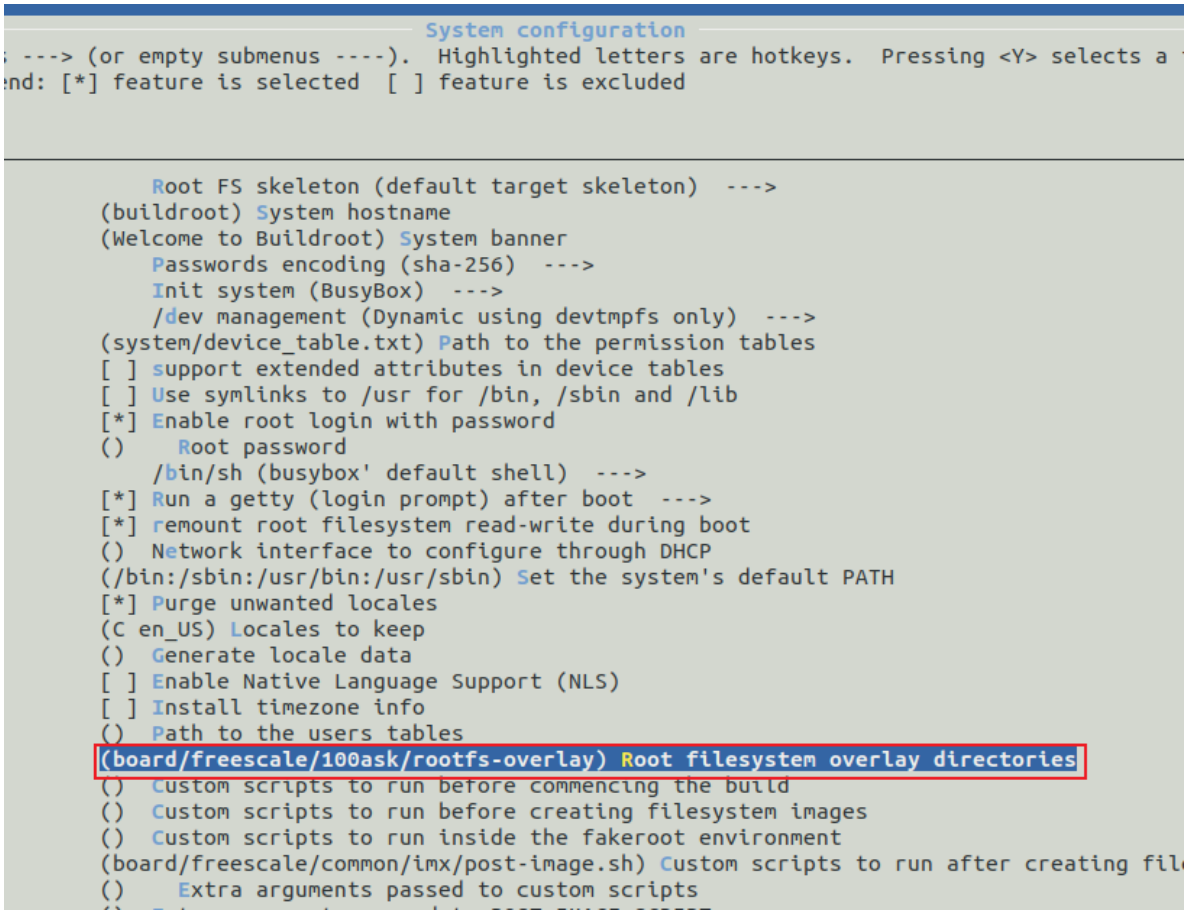
增加文件：

```
mkdir -p  board/freescale/100ask/rootfs-overlay/boot

cp ~/100ask_imx6ull-sdk/Linux-4.9.88/arch/arm/boot/zImage
board/freescale/100ask/rootfs-overlay/boot

cp ~/100ask_imx6ull-sdk/Linux-4.9.88/arch/arm/boot/dts/100ask_imx6ull-14x14.dtb
board/freescale/100ask/rootfs-overlay/boot
```

重新配置让`BR2_ROOTFS_OVERLAY`等于`board/freescale/100ask/rootfs-overlay/`



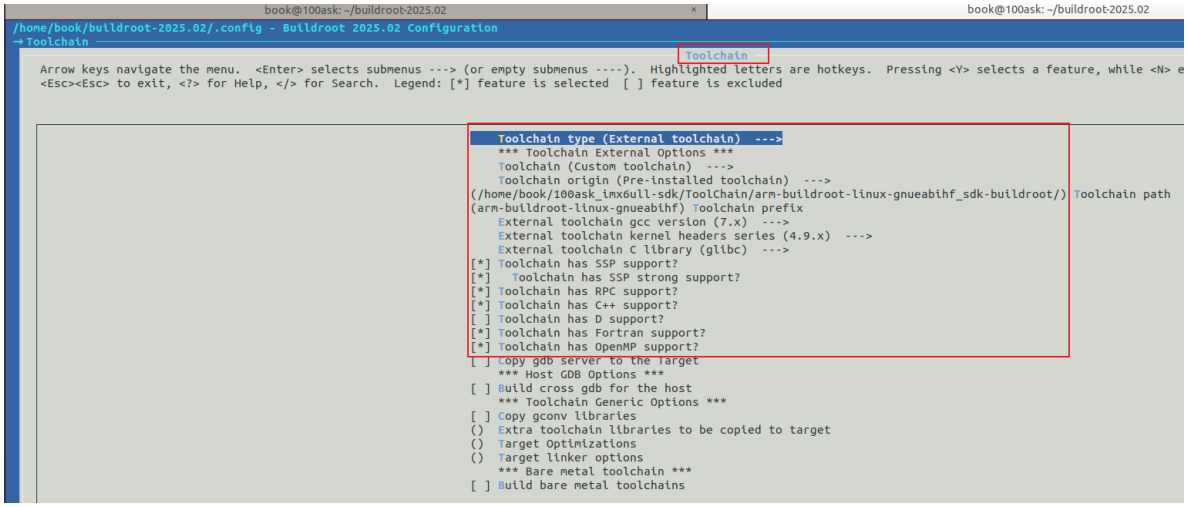
3.4 重新制作

重新执行`make`，即可得到`output/images/emmc.img`

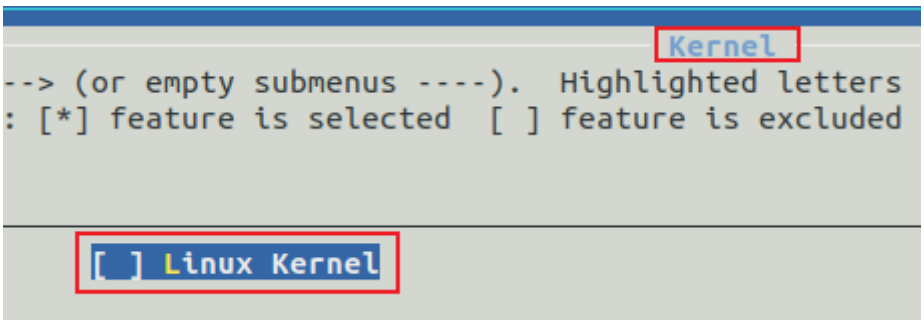
4. 使用外部工具链编译根文件系统

```
wget https://buildroot.org/downloads/buildroot-2025.02.tar.xz
tar xJf buildroot-2025.02.tar.xz
cd buildroot-2025.02
ls configs/*imx6ull*
make freescale_imx6ullevk_defconfig
make menuconfig # 选择外部工具链, 去掉内核、uboot
make -j 4
```

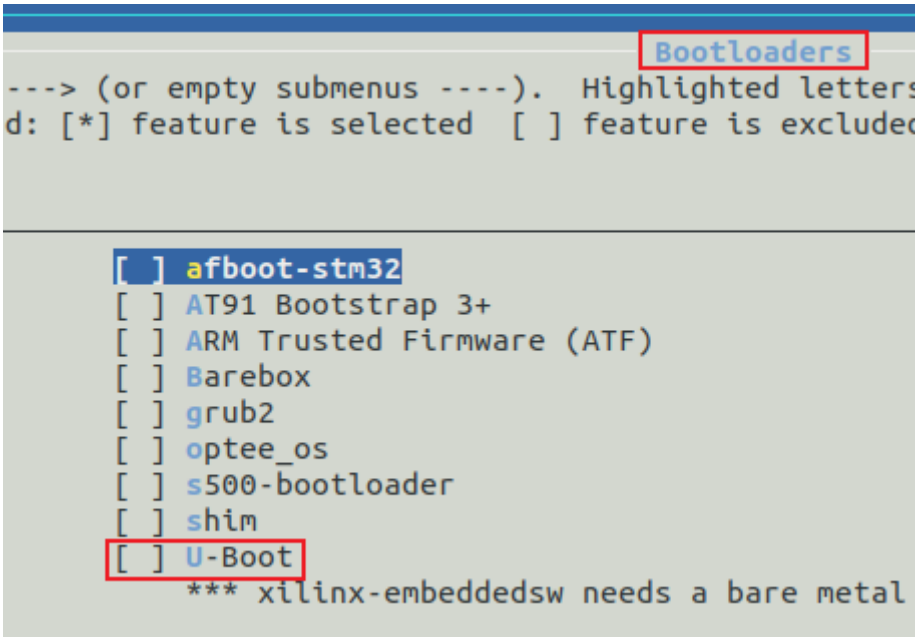
选择外部工具链：



去掉内核：



去掉uboot：



5. 添加开源APP、依赖库

5.1 添加Buildroot已经支持的软件

在`package`目录下有openocd、libmodbus，可以通过`make menuconfig`把它们加入：

```
make menuconfig
搜openocd配置上去
搜libmodbus配置上去
```

5.2 把本地Makefile工程加入Buildroot

参考：

- https://blog.csdn.net/david_liufei/article/details/135553702
- https://buildroot.org/downloads/manual/manual.html
  - 《Chapter 16. Coding style》
  - 《Chapter 18. Adding new packages to Buildroot》

5.2.1 工程源码

hello_make是一个简单的Makefile工程，内容如下：

```
$ cat hello_make.c
#include <stdio.h>

int main(int argc, char **argv)
{
        printf("hello, I am a make project\n");
        return 0;
}

$ cat Makefile
hello_make: hello_make.c
        $(CC) -o $@ $^

clean:
        rm -f hello_make
```

使用如下方法把它加入Buildroot。

5.2.2 在package目录下创建新的包目录

```
mkdir  package/hello_make
cd package/hello_make
touch hello_make.mk Config.in
```

hello_make.mk内容如下：

```
################################################################################
#
# hello_make
#
################################################################################
```

```
HELLO_MAKE_SOURCE =hello_make.tar.gz
HELLO_MAKE_SITE =$(TOPDIR)/dl/hello_make

define HELLO_MAKE_BUILD_CMDS
    $(MAKE) CC="$(TARGET_CC)" LD="$(TARGET_LD)" -C $(@D)
endef

define HELLO_MAKE_INSTALL_TARGET_CMDS
    $(INSTALL) -D -m 0755 $(@D)/hello_make $(TARGET_DIR)/usr/bin
endef

$(eval $(generic-package))
```

Config.in内容如下：

```
config BR2_PACKAGE_HELLO_MAKE
    bool "hello_make"
    help
      hello_make for test.
```

5.2.3 在dl目录下放置文件

```
dl/hello_make/hello_make.tar.gz
```

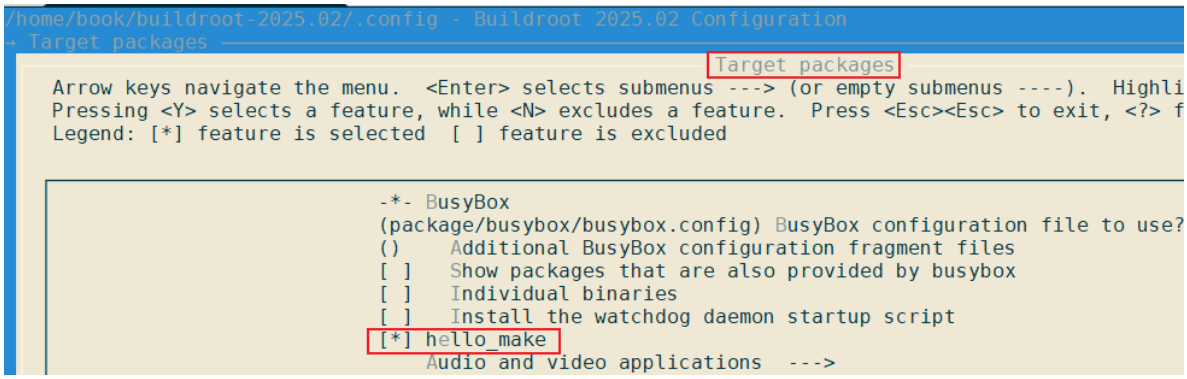
5.2.4 修改package/Config.mk

```
menu "Target packages"

        source "package/busybox/Config.in"
        source "package/skeleton/Config.in"
        source "package/skeleton-custom/Config.in"
        source "package/skeleton-init-common/Config.in"
        source "package/skeleton-init-none/Config.in"
        source "package/skeleton-init-openrc/Config.in"
        source "package/skeleton-init-systemd/Config.in"
        source "package/skeleton-init-sysv/Config.in"
        source "package/hello_make/Config.in"
```

5.2.5 配置

```
make menuconfig
```



5.2.6 编译

```
# 单独编译
make   hello_make

# 或则执行羡慕命令编译并且制作一系列映像文件
make
```

5.3 把本地cmake工程加入Buildroot

5.3.1 工程源码

hello_cmake是一个简单的Makefile工程，内容如下：

```
$ cat hello_cmake.c
#include <stdio.h>

int main(int argc, char **argv)
{
        printf("hello, I am a cmake project\n");
        return 0;
}


$ cat CMakeLists.txt
# 最低CMake版本要求
cmake_minimum_required(VERSION 3.10)

# 定义工程名称和版本
project(hello_cmake VERSION 1.0.0)

# 指定交叉编译工具链路径
#set(CMAKE_C_COMPILER arm-buildroot-linux-gnueabihf-gcc)
#set(CMAKE_CXX_COMPILER arm-buildroot-linux-gnueabihf-g++)

# 添加可执行文件
add_executable(hello_cmake hello_cmake.c)

# 可选：安装到系统路径（如/usr/local/bin）
install(TARGETS hello_cmake DESTINATION bin)
```

使用如下方法把它加入Buildroot。

5.2.2 在package目录下创建新的包目录

```
mkdir  package/hello_cmake
cd package/hello_cmake
touch hello_cmake.mk Config.in
```

hello_cmake.mk内容如下：

```
################################################################################
#
# hello_cmake
#
################################################################################

HELLO_CMAKE_SOURCE =hello_cmake.tar.bz2
HELLO_CMAKE_SITE =$(TOPDIR)/dl/hello_cmake


$(eval $(cmake-package))
```

Config.in内容如下：

```
config BR2_PACKAGE_HELLO_CMAKE
        bool "hello_cmake"
        help
          hello_cmake for test.
```

5.2.3 在dl目录下放置文件

```
dl/hello_cmake/hello_cmake.tar.bz2
```

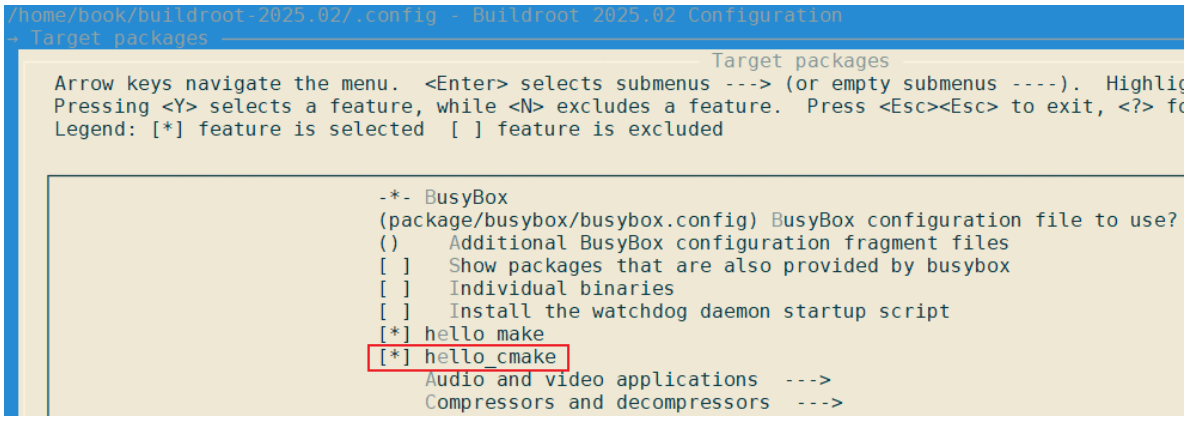
5.2.4 修改package/Config.mk

```
menu "Target packages"

        source "package/busybox/Config.in"
        source "package/skeleton/Config.in"
        source "package/skeleton-custom/Config.in"
        source "package/skeleton-init-common/Config.in"
        source "package/skeleton-init-none/Config.in"
        source "package/skeleton-init-openrc/Config.in"
        source "package/skeleton-init-systemd/Config.in"
        source "package/skeleton-init-sysv/Config.in"
        source "package/hello_make/Config.in"
        source "package/hello_cmake/Config.in"
```

5.2.5 配置

```
make menuconfig
```



5.2.6 编译

```
# 单独编译
make   hello_make

# 或则执行羡慕命令编译并且制作一系列映像文件
make
```